SIMATS ENGINEERING

ASSESSMENT TOOL 4

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA1711**

COURSE OUTCOME COVERED

CO2: Experiment search and game-solving techniques such as Greedy Search, A, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 40%

QUESTIONS: 3 **Duration: 1 Hour**

Total Marks:30

Annexure A

Dry Run Evaluation

Code of Conduct:

I Arulmozhi Chozhan - 1925123344(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

ASSESSMENT TOOL 4 — DRY RUN EVALUATION

COURSE NAME: Artificial Intelligence | COURSE CODE: CSA1711

Question 1 — Breadth-First Search (BFS)

Graph:

```
        A
       / \
      B   C
     / \ / \
    D  E F  G
```

BFS Trace Table (starting from A):

Iteration	Queue (after processing)	Visited Node
1	B, C	**A**
2	C, D, E	**B**
3	D, E, F, G	**C**
4	E, F, G	**D**
5	F, G	**E**
6	G	**F**
7	(empty)	**G**

Answer 1: BFS traversal order: A → B → C → D → E → F → G

Answer 2: Queue contents after each iteration: {B,C} → {C,D,E} → {D,E,F,G} → {E,F,G} → {F,G} → {G} → {} (as shown in the table above).

Question 2 — Depth-First Search (DFS)

Using the same graph, performing DFS starting from A (using a stack, expanding the left child first):

Iteration	Stack (after processing)	Visited Node
1	C, B	**A**
2	C, E, D	**B**
3	C, E	**D**
4	C	**E**
5	G, F	**C**
6	G	**F**
7	(empty)	**G**

Answer (DFS): DFS traversal order: A → B → D → E → C → F → G

Note: DFS goes as deep as possible down one branch (A → B → D) before backtracking to explore the next unvisited branch (E), then backtracks fully to explore C's subtree (F, G).

Question 3 — 8-Puzzle: Breadth-First Search

Initial State:

```
1 3 6
5 0 2
4 7 8
```

Goal State:

```
1 2 3
4 5 6
7 8 0
```

Iter.	Move Applied	Current State (Node Expanded)	Valid Moves from this State (Children Generated / Added to Queue)
0	— (Initial)	`1 3 6 / 5 0 2 / 4 7 8`	Up, Down, Left, Right
1	Right	`1 3 6 / 5 2 0 / 4 7 8`	Up, Down, Left
2	Up	`1 3 0 / 5 2 6 / 4 7 8`	Down, Left
3	Left	`1 0 3 / 5 2 6 / 4 7 8`	Down, Left, Right
4	Down	`1 2 3 / 5 0 6 / 4 7 8`	Up, Down, Left, Right
5	Left	`1 2 3 / 0 5 6 / 4 7 8`	Up, Down, Right
6	Down	`1 2 3 / 4 5 6 / 0 7 8`	Up, Right
7	Right	`1 2 3 / 4 5 6 / 7 0 8`	Up, Left, Right
8	Right	**`1 2 3 / 4 5 6 / 7 8 0 (GOAL)`**	— (goal test succeeds)

Answers to Sub-Questions

Answer 1: The node expanded at each iteration is the state shown in the "Current State" column of the table above — i.e. the sequence: (1 3 6/5 0 2/4 7 8) → (1 3 6/5 2 0/4 7 8) → (1 3 0/5 2 6/4 7 8) → (1 0 3/5 2 6/4 7 8) → (1 2 3/5 0 6/4 7 8) → (1 2 3/0 5 6/4 7 8) → (1 2 3/4 5 6/0 7 8) → (1 2 3/4 5 6/7 0 8) → (1 2 3/4 5 6/7 8 0).

Answer 2: Along the shortest path itself, 9 states are visited (8 expansions). However, a true breadth-first search explores every branch level by level before reaching depth 8: an exhaustive BFS of this puzzle expands 311 nodes and generates 838 child states in total (500 of which are distinct) before the goal state is dequeued. Only 8 of these moves lie on the final optimal solution path — the rest are alternative branches BFS must explore first because it processes all shallower nodes before any deeper one.

Answer 3: Complete solution path (8 moves):

- **Initial:** `1 3 6 / 5 0 2 / 4 7 8`
- **Move 1 (Right):** `1 3 6 / 5 2 0 / 4 7 8`
- **Move 2 (Up):** `1 3 0 / 5 2 6 / 4 7 8`
- **Move 3 (Left):** `1 0 3 / 5 2 6 / 4 7 8`
- **Move 4 (Down):** `1 2 3 / 5 0 6 / 4 7 8`
- **Move 5 (Left):** `1 2 3 / 0 5 6 / 4 7 8`
- **Move 6 (Down):** `1 2 3 / 4 5 6 / 0 7 8`
- **Move 7 (Right):** `1 2 3 / 4 5 6 / 7 0 8`
- **Move 8 (Right):** **`1 2 3 / 4 5 6 / 7 8 0 ✓ Goal`**

Move sequence: Right → Up → Left → Down → Left → Down → Right → Right

Answer 4: BFS always finds the shortest solution in an unweighted 8-puzzle problem because it expands nodes in strict order of increasing depth — it fully explores every state reachable in d moves before exploring any state reachable in d+1 moves. Since every move (tile slide) has the same cost (1), the first time BFS dequeues the goal state is guaranteed to be via the minimum possible number of moves; no shorter path could exist, because such a path would have been found at an earlier (shallower) level of the search before the goal was ever reached at depth 8.